

Session 6

Functional Programming using **map()**

1. Build Array from Permutation- <https://leetcode.com/problems/build-array-from-permutation/>

Given a **zero-based permutation** **nums** (**0-indexed**), build an array **ans** of the **same length** where **ans[i] = nums[nums[i]]** for each $0 \leq i < \text{nums.length}$ and return it.

A **zero-based permutation** **nums** is an array of **distinct** integers from 0 to **nums.length - 1** (**inclusive**).

Example 1:

Input: **nums** = [0,2,1,5,3,4]

Output: [0,1,2,4,5,3]

Explanation: The array **ans** is built as follows:

```
ans = [nums[nums[0]], nums[nums[1]], nums[nums[2]], nums[nums[3]], nums[nums[4]],  
nums[nums[5]]]
```

```
= [nums[0], nums[2], nums[1], nums[5], nums[3], nums[4]]
```

```
= [0,1,2,4,5,3]
```

Example 2:

Input: **nums** = [5,0,1,2,3,4]

Output: [4,5,0,1,2,3]

Explanation: The array **ans** is built as follows:

```
ans = [nums[nums[0]], nums[nums[1]], nums[nums[2]], nums[nums[3]], nums[nums[4]],  
nums[nums[5]]]
```

```
= [nums[5], nums[0], nums[1], nums[2], nums[3], nums[4]]
```

```
= [4,5,0,1,2,3]
```

Program

```
class Solution {  
  
    public int[] buildArray(int[] nums) {  
  
        int[] ans = new int[nums.length];  
  
        for (int i = 0; i < nums.length; i++) {  
  
            ans[i] = nums[nums[i]];
```

```

    }

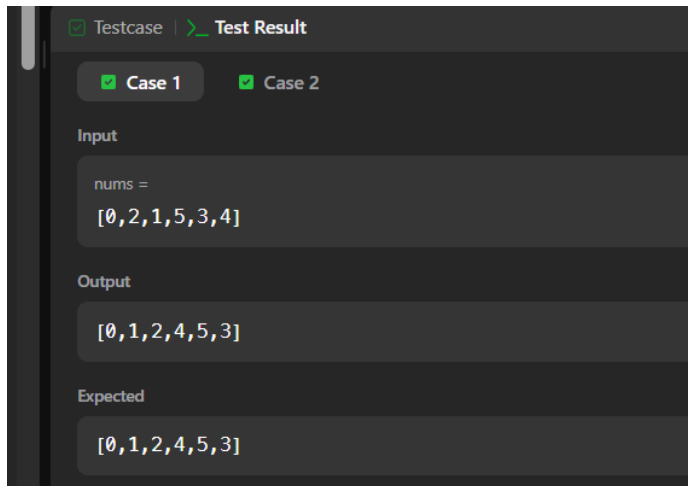
    return ans;

}

}

```

Output



2. Shuffle the Array – <https://leetcode.com/problems/shuffle-the-array/>

Given the array `nums` consisting of $2n$ elements in the form $[x_1, x_2, \dots, x_n, y_1, y_2, \dots, y_n]$.

Return the array in the form $[x_1, y_1, x_2, y_2, \dots, x_n, y_n]$.

Example 1:

Input: `nums = [2,5,1,3,4,7]`, `n = 3`

Output: `[2,3,5,4,1,7]`

Explanation: Since $x_1=2$, $x_2=5$, $x_3=1$, $y_1=3$, $y_2=4$, $y_3=7$ then the answer is `[2,3,5,4,1,7]`.

Example 2:

Input: `nums = [1,2,3,4,4,3,2,1]`, `n = 4`

Output: `[1,4,2,3,3,2,4,1]`

Example 3:

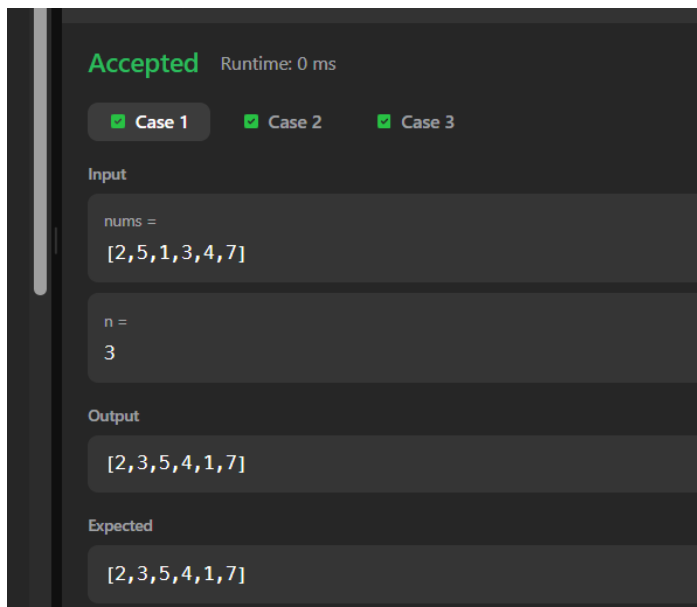
Input: `nums = [1,1,2,2]`, `n = 2`

Output: `[1,2,1,2]`

Program

```
class Solution {  
    public int[] shuffle(int[] nums, int n) {  
        int[] ans = new int[2 * n];  
        int j = 0;  
        for (int i = 0; i < n; i++) {  
            ans[j++] = nums[i];  
            ans[j++] = nums[i + n];  
        }  
        return ans;  
    }  
}
```

Output



Session 8

2. Find the Highest Altitude – <https://leetcode.com/problems/find-the-highest-altitude/>

There is a biker going on a road trip. The road trip consists of $n + 1$ points at various altitudes. The biker starts his trip on point 0 with altitude equal 0.

You are given an integer array `gain` of length n where `gain[i]` is the **net gain in altitude** between points i and $i + 1$ for all $(0 \leq i < n)$. Return *the **highest altitude** of a point*.

Example 1:

Input: `gain = [-5,1,5,0,-7]`

Output: 1

Explanation: The altitudes are `[0,-5,-4,1,1,-6]`. The highest is 1.

Example 2:

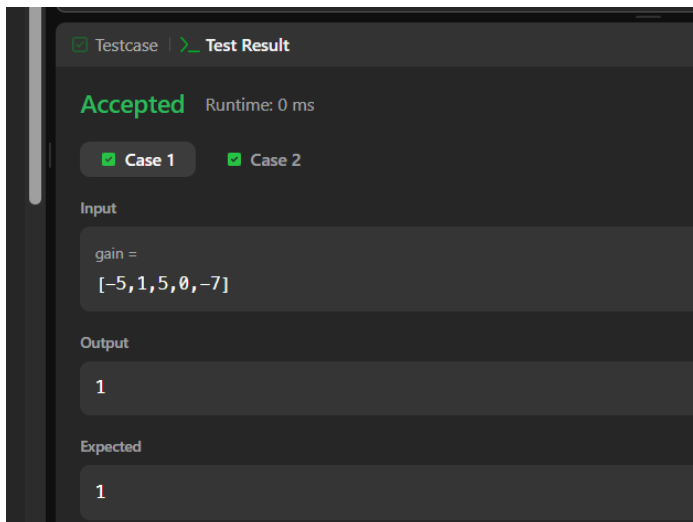
Input: `gain = [-4,-3,-2,-1,4,3,2]`

Output: 0

Explanation: The altitudes are `[0,-4,-7,-9,-10,-6,-3,-1]`. The highest is 0.

Program

```
class Solution {  
    public int largestAltitude(int[] gain) {  
        int altitude = 0;  
        int highest = 0;  
        for (int i = 0; i < gain.length; i++) {  
            altitude = altitude + gain[i];  
            if (altitude > highest) {  
                highest = altitude;  
            }  
        }  
        return highest;  
    }  
}
```



Session 10

Stream API & Collection Processing

1. Java Dequeue – <https://www.hackerrank.com/challenges/java-dequeue/problem>

In computer science, a double-ended queue (dequeue, often abbreviated to deque, pronounced deck) is an abstract data type that generalizes a queue, for which elements can be added to or removed from either the front (head) or back (tail).

Deque interfaces can be implemented using various types of collections such as `LinkedList` or `ArrayDeque` classes. For example, deque can be declared as:

```
Deque deque = new LinkedList<>();  
or  
Deque deque = new ArrayDeque<>();
```

You can find more details about Deque [here](#).

In this problem, you are given `n` integers. You need to find the maximum number of unique integers among all the possible contiguous subarrays of size `k`.

Note: Time limit is 1 second for this problem.

Input Format

The first line of input contains two integers `n` and `k`: representing the total number of integers and the size of the subarray, respectively. The next line contains `n` space separated integers.

Constraints

The numbers in the array will range between .

Output Format

Print the *maximum* number of unique integers among all possible contiguous subarrays of size .

Sample Input

```
6 3
5 3 5 2 3 2
```

Sample Output

```
3
```

Explanation

In the sample testcase, there are 4 subarrays of contiguous numbers.

- Has unique numbers.
- Has unique numbers.
- Has unique numbers.
- Has unique numbers.

In these subarrays, there are unique numbers, respectively. The maximum amount of unique numbers among all possible contiguous subarrays is .

Program

```
import java.util.*;

public class test {

    public static void main(String[] args) {

        Scanner in = new Scanner(System.in);

        Deque<Integer> deque = new ArrayDeque<>();

        Set<Integer> set = new HashSet<>();

        int n = in.nextInt();

        int m = in.nextInt();
```

```

int maxUnique = 0;

for (int i = 0; i < n; i++) {

    int num = in.nextInt();

    // Add current number to deque and set

    deque.addLast(num);

    set.add(num);

    // Once deque reaches window size m

    if (deque.size() == m) {

        // Update max unique numbers found so far

        maxUnique = Math.max(maxUnique, set.size());

        // Early exit optimization: maximum possible unique numbers in a window of size m is m

        if (maxUnique == m) {

            System.out.println(maxUnique);

            return;

        // Remove the oldest element from the left side of the window

        int removed = deque.removeFirst();

        // If the removed element is no longer in the deque, remove it from the set

        if (!deque.contains(removed)) {

            set.remove(removed);

        }

    }

    System.out.println(maxUnique);

    in.close();

}

}

```

Output

Congratulations!
You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✔ Sample Test case 0

Input (stdin)

1 6 3
2 5 3 5 2 3 2

Your Output (stdout)

1 3

Expected Output

1 3

2. Java Hashset – <https://www.hackerrank.com/challenges/java-hashset/problem>

In computer science, a set is an abstract data type that can store certain values, without any particular order, and no repeated values(Wikipedia). is an example of a set, but is not a set. Today you will learn how to use sets in java by solving this problem.

You are given pairs of strings. Two pairs and are identical if and . That also implies is *not* same as . After taking each pair as input, you need to print number of unique pairs you currently have.

Complete the code in the editor to solve this problem.

Input Format

In the first line, there will be an integer denoting number of pairs. Each of the next lines will contain two strings seperated by a single space.

Constraints:

- Length of each string is atmost and will consist lower case letters only.

Output Format

Print lines. In the line, print number of unique pairs you have after taking pair as input.

Sample Input

5

john tom

john mary

john tom

mary anna

mary anna

Sample Output

1

2

2

3

3

Explanation

- After taking the first input, you have only one pair: (john,tom)
- After taking the second input, you have two pairs: (john, tom) and (john, mary)
- After taking the third input, you still have two unique pairs.
- After taking the fourth input, you have three unique pairs: (john,tom), (john, mary) and (mary, anna)
- After taking the fifth input, you still have three unique pairs.

program

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

    public static void main(String[] args) {

        Scanner s = new Scanner(System.in);

        int t = s.nextInt();

        String [] pair_left = new String[t];

        String [] pair_right = new String[t];
```

```
for (int i = 0; i < t; i++) {  
  
    pair_left[i] = s.next();  
  
    pair_right[i] = s.next();  
  
}
```

//Write your code here

```
HashSet<String> set = new HashSet<>();
```

```
for (int i = 0; i < t; i++) {  
  
    String pair = pair_left[i] + " " + pair_right[i];  
  
    set.add(pair);  
  
    System.out.println(set.size());  
  
}  
  
}  
  
}
```

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✔ Sample Test case 0

Input (stdin)

```
1 5  
2 john tom  
3 john mary  
4 john tom  
5 mary anna  
6 mary anna
```

Your Output (stdout)

```
1 1  
2 2  
3 2
```